Pack 04 · Measure and operate

Serving metrics, load testing, percentiles, SLOs and capacity planning. Lessons 01, 06, 08, 15, 16, 20–25, 27, 30, 43. Answer each question out loud in about a minute, then check the card.

Q1 · "Request X is 4× slower" PRIORITY

Lessons: 08, 16, 20

> **INTERVIEWER**
>
> Request X took 8.3 s end to end with 400 output tokens; request Y took 2.0 s with 85. Both show TTFT 300 ms and TPOT 20 ms. The PM says X is 4× slower. Is it, and what should the PM compare?

60-second answer

- E2E ≈ TTFT + TPOT × (N − 1): X = 0.3 + 399 × 0.02 ≈ 8.3 s, Y = 0.3 + 84 × 0.02 ≈ 2.0 s.
- Same TTFT, same TPOT: **identical serving speed**. X just wrote more tokens.
- Compare **TTFT** (wait for the first token) and **TPOT** (speed of the stream), not E2E across different lengths.
- TPOT = (E2E − TTFT) ÷ (N − 1): there are N − 1 gaps after the first token.

Key numbers: TPOT = (E2E − TTFT) ÷ (N − 1); e.g. (2.3 − 0.3) s ÷ 400 = 5 ms for 401 tokens.

Likely follow-up: "Average TPOT looks fine but users say the stream stutters?" Look at the **distribution of inter-token gaps** (p95, p99, max), not the mean. Long prefills of other requests are the usual cause.

> **TRAP**
>
> I divided by N instead of N − 1, and blamed stutter on queue length (that explains slow starts, not uneven gaps).

Q2 · Two tools, two TPOTs PRIORITY

Lessons: 20

> **INTERVIEWER**
>
> A load test ran 50 short chats (20 tokens, 50 ms gaps) and 10 long answers (1,000 tokens, 10 ms gaps). The dashboard says TPOT ≈ 13.5 ms; a teammate's script says ≈ 43 ms. Which is right for "what a typical user feels"? And after turning on speculative decoding, a client script reports 45 ms per message with 3 tokens per message.

60-second answer

- Per request (one vote per request): (50 × 50 + 10 × 10) ÷ 60 ≈ **43 ms**.

- Per token (every gap counts): (50 × 19 × 50 + 10 × 999 × 10) ÷ (950 + 9,990) ≈ **13.5 ms**, dominated by the 10 long answers.
- "Typical user" is **per request**: 50 of 60 users felt 50 ms.
- Speculative decoding sends tokens in chunks: 45 ms ÷ 3 = **15 ms per token**. The script timed messages, not tokens.

Key numbers: per-token TPOT weights long requests by their gap count.

Likely follow-up: "Which would you put in an SLO?" Per-request percentiles (e.g. p99 per-request TPOT), so every user counts once.

> **TRAP**
> I mixed gap counts into the per-request vote and forgot to multiply by the number of requests in the per-token sum.

Q3 · Prefill speed from a TTFT sweep PRIORITY

Lessons: 15, 24

> **INTERVIEWER**
> Single-request TTFT, max_tokens = 1, medians: 256 tokens → 23 ms, 8,000 tokens → 280 ms. Model TTFT = fixed cost + prompt ÷ throughput. What are the prefill throughput and fixed cost, and what's in the fixed cost?

60-second answer

- Throughput is the **slope**: (8,000 − 256) ÷ (280 − 23) ms ≈ 7,744 ÷ 0.257 s ≈ **30k tokens/s**.
- Fixed cost is the **intercept**: 23 ms − 256 ÷ 30k ≈ 23 − 8.5 ≈ **14.5 ms**.
- The fixed cost: HTTP and tokenization, scheduling, the first decode step, detokenizing and streaming the first token.

Key numbers: throughput = Δtokens ÷ ΔTTFT.

Likely follow-up: "Why not divide 256 by 23 ms?" That folds the fixed cost into the rate and gives ≈ 11k tokens/s, almost 3× too low.

> **TRAP**
> I divided one point. Use two points: slope for the rate, intercept for the overhead.

Q4 · "The benchmark said we're fine" PRIORITY

Lessons: 21, 25

> **INTERVIEWER**
>
> A closed-loop benchmark with 32 concurrent users measured 4 s mean time in system. At launch, real users arrive at 10 req/s. What happens? Which load test should you have run, and why can a waiting load generator hide a 3 s stall?

60-second answer

- **Little's law**: throughput = in-flight ÷ time = 32 ÷ 4 s = **8 req/s**. At 10 req/s arrivals the queue grows by 2 req/s, **120 waiting after a minute**, without bound.
- A **closed loop** slows its own arrivals when the server slows, so it can't show overload.
- Max throughput: **closed-loop concurrency sweep**. Latency at launch traffic: **open-loop Poisson arrivals at 10 req/s**.
- **Coordinated omission**: a client that waits for each response sends ≈ 1 request during a 3 s stall instead of 15, so p99 never sees it. A scheduled client records all 15 of 300 (5%), and p99 ≈ 2.6 s.

Key numbers: $L = \lambda \times W$; queue growth = arrivals − completions.

Likely follow-up: "Traffic grows 30% on a server at 70% busy?" $\rho = 0.91$; time in system $\propto 1 \div (1 - \rho)$: 11.1 ÷ 3.3 ≈ **3.3× longer**. Near saturation, small traffic increases blow up latency.

> **TRAP**
>
> I explained why the test missed it but not what happens (compute the growth). For max throughput, sweep concurrency, not request rate.

Q5 · Percentile traps PRIORITY

Lessons: 06, 22

> **INTERVIEWER**
>
> Three dashboards: P99 TTFT doubled from 0.8 to 1.5 s with no deploy and unchanged per-request timings; ops averages four replicas' P99s (0.6, 0.7, 0.6, 3.0 s) into "fleet P99 = 1.2 s"; and an agent makes 20 sequential calls against an API with P99 = 2 s. What's wrong in each?

60-second answer

- **Mix shift**: more long prompts raise blended P99 TTFT without any regression. Segment by prompt length and compare each segment and its traffic share.
- **Percentiles don't average**: merge histograms, then take the percentile. The slowest 1% of the fleet is ≈ 4% of the bad replica, i.e. its **P96**, likely ≈ 2 s, so the truth is **well above 1.2 s**.
- **Tails compound**: P(at least one slow call) = $1 - 0.99^{20}$ ≈ **18%** of tasks. For 1% of tasks to be slow, each call needs ≈ **P99.95**.

Key numbers: $1 - (1 - q)^k \approx k \times q$ for small q.

Likely follow-up: "How many samples does a P99 need?" About 100 samples beyond the percentile for ≈ 10% precision, so ≈ 10,000.

> **TRAP**
>
> I said "load increased" (that would show as queueing) and blamed response length for TTFT (prompt length drives TTFT). For the agent I read "1%" as per task.

Q6 · Is a 3.5% speedup real? PRIORITY

Lessons: 06

> **INTERVIEWER**
>
> Before: 5 runs averaging 20.0 ms TPOT. After a tweak: 5 runs averaging 19.3 ms. Each run wobbles by σ ≈ 0.8 ms. A teammate says "3.5% faster, ship it."

60-second answer

- One run wobbles σ; an average of n wobbles **σ ÷ √n** = 0.8 ÷ √5 ≈ 0.36 ms.
- A difference of two averages wobbles **√2** more ≈ 0.51 ms, so the 95% band is ≈ **±1.0 ms**.
- 0.7 ms is inside the noise: **not proven**.
- To resolve 0.7 ms you need about 2 × 0.8 × √2 ÷ √n ≤ 0.7, so **n ≈ 11+ runs each**, or pair the runs to cancel shared noise.

Key numbers: 95% band on a difference ≈ ±2 × √2 × σ ÷ √n.

Likely follow-up: "Same idea for model quality?" Yes: compare question by question (paired), as in the quantization pack.

> **TRAP**
>
> I had no mental model to start from: σ, then σ ÷ √n, then × √2 for a difference, then a gap beyond ≈ 2× that.

Q7 · Goodput and the operating point PRIORITY

Lessons: 23, 25, 27, 43

> **INTERVIEWER**
>
> SLO: TTFT < 500 ms and TPOT < 30 ms. At 6 req/s, 5% fail with 429; of the successes 90% meet TTFT and 85% meet TPOT. What's the goodput? And a sweep peaks at 5,000 tokens/s, but every user must get at least 100 tokens/s at P99. What do you put in the capacity doc?

60-second answer

- Goodput = 6 × 0.95 × 0.90 × 0.85 ≈ **4.4 req/s**, only **73%** of the dashboard's 6.

- 100 tokens/s at P99 means P99 TPOT ≤ **10 ms**. Pick the **highest concurrency that still passes** (32 users, ≈ 3,400 tokens/s), never the 5,000 peak.
- Capacity = **min(saturation wall, SLO wall, memory wall)**.
- The fastest config can lose: big steps break TPOT first, a low admission cap breaks TTFT first. Pick by **SLO-safe** throughput.

Key numbers: goodput = rate × success rate × product of SLO pass rates.

Likely follow-up: "What's the memory wall?" Pool tokens ÷ final tokens per request; for long agent outputs it's often the binding one.

> **TRAP**
> I picked the point "closest to" the target and leaned toward the peak. "At least" means the rightmost point that still passes.

Q8 · "40 ÷ 8 = 5 GPUs" PRIORITY

Lessons: 43

> **INTERVIEWER**
> The busiest minute averages 40 req/s and one H100 holds the SLO up to 8 req/s. The manager says 5 GPUs. With a 20% burst margin and surviving one replica failure, how many? And when traffic exceeds plan, should you raise max_num_seqs from the tested 128 to 512?

60-second answer

- Usable per GPU = 8 × 0.8 = 6.4 req/s; 40 ÷ 6.4 = 6.25, so **7** GPUs serve the load.
- Surviving one loss: N − 1 ≥ 7, so **8 GPUs**.
- Don't raise the cap: users already streaming get slower and **preemptions** start.
- Protect the operating point instead: keep the tested cap, **time out or reject** excess queueing (429), and **autoscale early**.

Key numbers: N − 1 ≥ peak ÷ (per-GPU SLO capacity × (1 − margin)).

Likely follow-up: "Why not VTC for overload?" VTC balances tenants; it doesn't shed load.

> **TRAP**
> I answered "N + 2" from memory instead of working the inequality.

Q9 · Should someone be paged? PRIORITY

Lessons: 23

> **INTERVIEWER**
>
> SLO: 99.5% good over 30 days, about 2 million requests a month. Over the last hour 3% of requests were bad. How big is the error budget, what's the burn rate, and how long until it's gone?

60-second answer

- Budget = 0.5% × 2 M = **10,000 bad requests** a month.
- Burn rate = 3% ÷ 0.5% = **6×**.
- At 6× the 30-day budget lasts 30 ÷ 6 = **5 days**. Page: that's a fast burn.

Key numbers: days left = 30 ÷ burn rate.

Likely follow-up: "How do you avoid paging on blips?" Alert on burn rate over two windows (e.g. 1 h and 5 min both high).

> **TRAP**
>
> I divided 6 by 30. Sanity check: at 1× the budget lasts exactly 30 days, so faster burns leave fewer days.

Q10 · Autoscaling on the wrong signal PRIORITY

Lessons: 01, 30

> **INTERVIEWER**
>
> An autoscaler adds a replica when CPU passes 70%. During a spike, users wait 5+ s for the first token but CPU shows 12%, so nothing scales. What should it watch, and how long does a new replica take with 16 GB of weights at 2 GB/s?

60-second answer

- CPU is idle in GPU serving, and **GPU memory** always reads ≈ 90% because the KV pool is preallocated.
- Scale on **engine signals**: requests waiting (queue growth), KV cache usage, and TTFT.
- Cold start ≥ 16 GB ÷ 2 GB/s = **8 s** plus engine startup (CUDA graphs, warm-up).
- So **trigger early**, keep a **warm pool**, use faster storage, or load fewer bytes (quantized weights).

Key numbers: cold start ≥ weight bytes ÷ load bandwidth.

Likely follow-up: "Which single metric would you pick?" `num_requests_waiting`: it rises before latency does.

> **TRAP**
>
> I first said GPU memory utilization, which never moves because of preallocation.

Build it yourself

- `exercises/06_08_09_tail_sim.py`: simulates how much a P99 estimate wobbles with sample size, and brute-forces the 20-call agent's 18%. Run `uv run exercises/06_08_09_tail_sim.py`.
- `exercises/24_27_find_walls.py`: finds the saturation, SLO and memory walls from a concurrency sweep and returns the binding capacity. Run `uv run exercises/24_27_find_walls.py`.